# COFFEE SALES ANALYSIS USING SQL

By: Aish Joon

Tools: MySQL | Excel

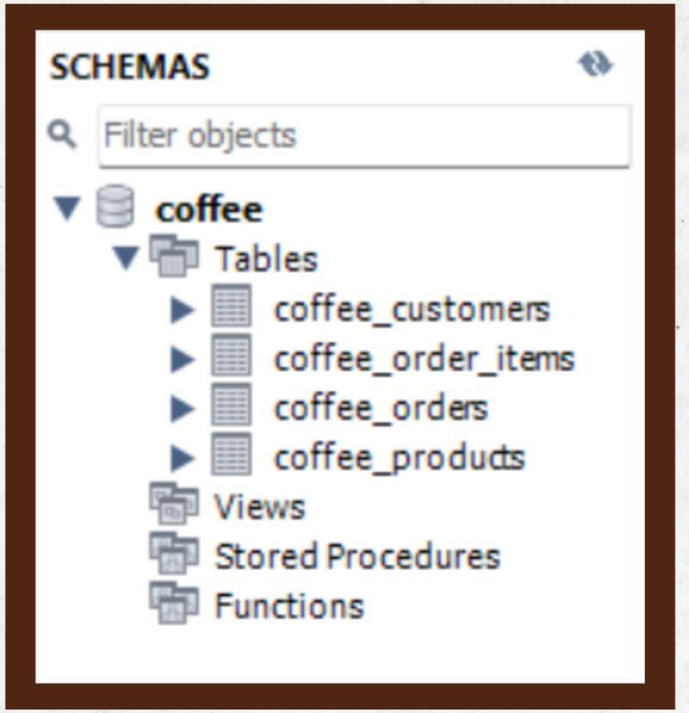
"Transforming data into decisions"





# PROJECT OVERVIEW

This project focuses on analyzing coffee sales data using SQL to uncover meaningful business insights related to customer behavior, product performance, and revenue trends. By working with structured datasets containing customer, order, and product information, the analysis aims to identify key patterns such as top-performing coffee types, high-value customers, and sales trends over time.



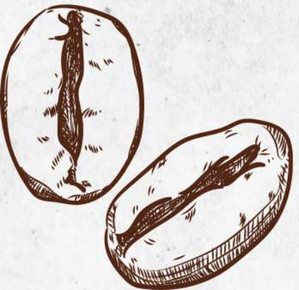
# BASIC: PROBLEM STATEMENT-1

Find the total revenue generated from coffee sales.

```
# Find the total revenue generated from coffee sales.
SELECT sum(quantity * amount) AS Revenue
FROM coffee_order_items;
```

**Output:**

| | Revenue |
|---|---|
| ▶ | 767160 |





# BASIC: PROBLEM STATEMENT-2

Find the total number of orders placed by each customer.

```sql
# Find the total number of orders placed by each customer.
SELECT c.customer_name, count(co.order_id) AS Total_Orders
FROM coffee_order_items AS coi
JOIN coffee_orders AS co
ON coi.order_id = co.order_id
JOIN coffee_customers AS c
ON c.customer_id = co.customer_id
GROUP BY c.customer_name
ORDER BY Total_Orders DESC;
```





# BASIC: PROBLEM STATEMENT-3

Find the total revenue generated by each coffee type.

```sql
# Find the total revenue generated by each coffee type.
SELECT coffee_type, sum(quantity * amount) AS Revenue
FROM coffee_order_items
GROUP BY coffee_type
ORDER BY Revenue DESC;
```

## Output:

| coffee_type | Revenue |
|---|---|
| Cold Brew | 137920 |
| Cappuccino | 129710 |
| Flat White | 128440 |
| Mocha | 122000 |
| Latte | 98100 |
| Americano | 90750 |
| Espresso | 60240 |

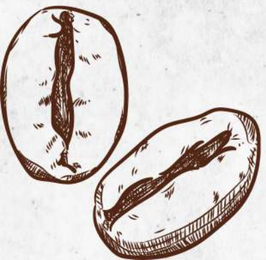




# BASIC: PROBLEM STATEMENT-4

Find the total quantity of coffee sold per day.

```
# Find the total quantity of coffee sold per day.
SELECT co.order_date, sum(coi.quantity) AS Total_Quantity
FROM coffee_order_items AS coi
JOIN coffee_orders AS co
ON coi.order_id = co.order_id
GROUP BY co.order_date
ORDER BY co.order_date ASC;
```





# BASIC: PROBLEM STATEMENT-5

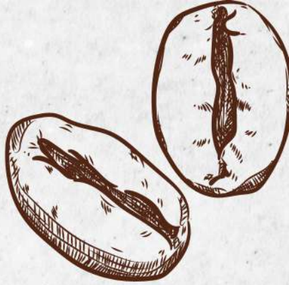
Find the top 10 customers who generated the highest revenue.

```sql
# Find the top 10 customers who generated the highest revenue.
SELECT c.customer_name, sum(coi.quantity * coi.amount) AS Revenue
FROM coffee_order_items AS coi
JOIN coffee_orders AS co
ON coi.order_id = co.order_id
JOIN coffee_customers AS c
ON c.customer_id = co.customer_id
GROUP BY c.customer_name
ORDER BY Revenue DESC
LIMIT 10;
```



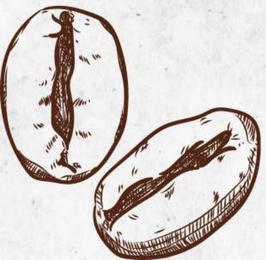


# INTERMEDIATE: PROBLEM STATEMENT-1



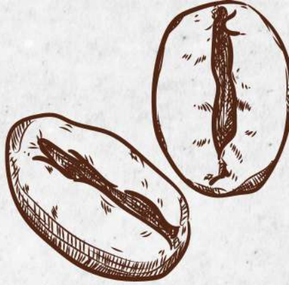
Find the coffee types whose total sales revenue is greater than 10K.

```sql
# Find the coffee types whose total sales revenue is greater than 10K.
SELECT coffee_type, sum(quantity * amount) AS Revenue
FROM coffee_order_items
GROUP BY coffee_type
HAVING Revenue > 10000
ORDER BY Revenue DESC;
```



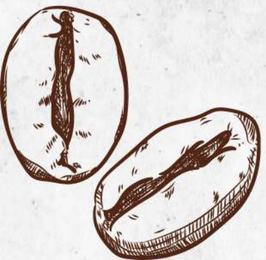


# INTERMEDIATE: PROBLEM STATEMENT-2



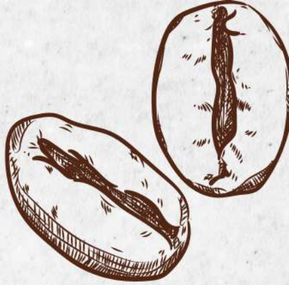
Find the customer who placed the highest number of orders.

```sql
# Find the customer who placed the highest number of orders.
SELECT c.customer_name, count(co.order_id) AS Total_Orders
FROM coffee_customers AS c
JOIN coffee_orders AS co
ON c.customer_id = co.customer_id
GROUP BY c.customer_name
ORDER BY Total_Orders DESC
LIMIT 1;
```



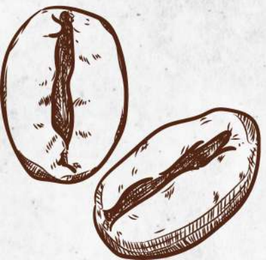


# INTERMEDIATE: PROBLEM STATEMENT-3



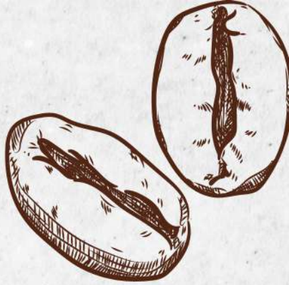
Find the customers who never placed an order.

```
# Find the customers who never placed an order.
SELECT c.customer_id, c.customer_name
FROM coffee_customers AS c
LEFT JOIN coffee_orders AS co
ON c.customer_id = co.customer_id
WHERE co.customer_id IS NULL;
```



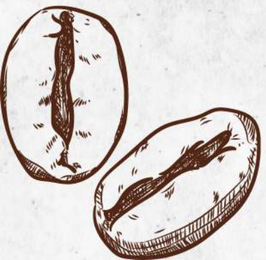


# INTERMEDIATE: PROBLEM STATEMENT-4



Find the orders where total amount spent is greater than the average order value.

```
# Find the orders where total amount spent is greater than the average order value.
SELECT order_id, sum(quantity * amount) AS Revenue
FROM coffee_order_items
GROUP BY order_id
HAVING Revenue > (SELECT AVG(quantity * amount)
                  FROM coffee_order_items);
```

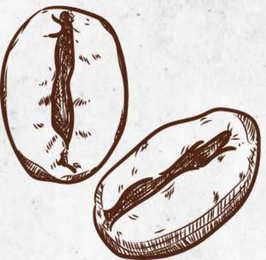




# INTERMEDIATE: PROBLEM STATEMENT-5

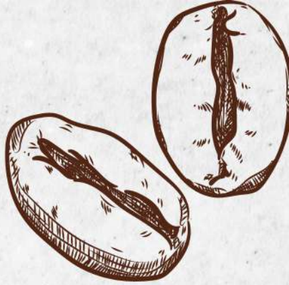
Find the month with the highest revenue.

```
# Find the month with the highest revenue.
SELECT monthname(co.order_date), sum(coi.quantity * coi.amount) AS Revenue
FROM coffee_orders AS co
JOIN coffee_order_items AS coi
ON co.order_id = coi.order_id
GROUP BY monthname(co.order_date)
ORDER BY Revenue DESC
LIMIT 1;
```



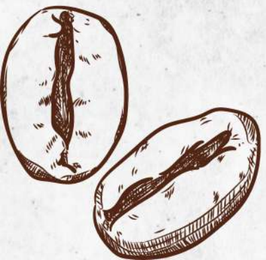


# ADVANCED: PROBLEM STATEMENT-1



Find the percentage contribution of each coffee type to total revenue.

```sql
# Find the percentage contribution of each coffee type to total revenue.
SELECT ifnull(coffee_type, "TOTAL"),
       SUM(amount * quantity) AS Total_Revenue,
       ROUND((SUM(amount * quantity) * 100.0) /
             (SELECT SUM(amount * quantity) FROM coffee_order_items), 2) AS Revenue_Percentage
FROM coffee_order_items
GROUP BY coffee_type with rollup
ORDER BY Revenue_Percentage DESC;
```

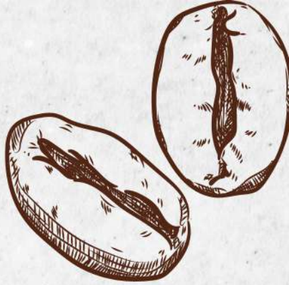


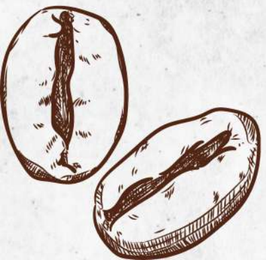


# ADVANCED: PROBLEM STATEMENT-2

Find the rank of coffee types based on total revenue generated.

```sql
# Find the rank of coffee types based on total revenue generated.
SELECT
    coffee_type,
    SUM(quantity * amount) AS Total_Revenue,
    RANK() OVER (ORDER BY SUM(quantity * amount) DESC) AS Revenue_Rank
FROM coffee_order_items
GROUP BY coffee_type;
```







# ADVANCED: PROBLEM STATEMENT-3

Find the top 3 coffee types generating the highest revenue each month.

```sql
# Find the top 3 coffee types generating the highest revenue each month (Window Functions)
SELECT *
FROM (
    SELECT
        monthname(co.order_date) AS Month, coi.coffee_type,
        SUM(coi.quantity * coi.amount) AS Total_Revenue,
        RANK() OVER (PARTITION BY monthname(co.order_date)
                     ORDER BY SUM(coi.quantity * coi.amount) DESC) AS rnk
    FROM coffee_orders AS co
    JOIN coffee_order_items AS coi
    ON co.order_id = coi.order_id
    GROUP BY Month, coi.coffee_type
) t
WHERE rnk <= 3;
```

# ADVANCED: PROBLEM STATEMENT-4

Find customers who ordered at least 3 different coffee types.

```sql
# Find customers who ordered at least 3 different coffee types.
SELECT c.customer_id, c.customer_name,
       COUNT(DISTINCT coi.coffee_type) AS Unique_Coffee_Types
FROM coffee_customers AS c
JOIN coffee_orders AS co
ON c.customer_id = co.customer_id
JOIN coffee_order_items AS coi
ON co.order_id = coi.order_id
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(DISTINCT coi.coffee_type) >= 3
ORDER BY Unique_Coffee_Types DESC;
```

# ADVANCED: PROBLEM STATEMENT-5

Find the top 2 customers in each city by revenue using window functions.

```sql
# Find the top 2 customers in each city by revenue using window functions.
SELECT city, customer_id, customer_name, Total_Revenue
FROM (
    SELECT c.city, c.customer_id, c.customer_name,
           SUM(coi.amount) AS Total_Revenue,
           ROW_NUMBER() OVER (PARTITION BY c.city
                              ORDER BY SUM(coi.amount) DESC) AS rnk
    FROM coffee_customers c
    JOIN coffee_orders co ON c.customer_id = co.customer_id
    JOIN coffee_order_items coi ON co.order_id = coi.order_id
    GROUP BY c.city, c.customer_id, c.customer_name
) t
WHERE rn <= 2
ORDER BY city, rnk;
```

# KEY INSIGHTS & CONCLUSIONS

1. Restaurant generated a total revenue of approximately $760K.
2. “Cold Brew” is the highest revenue generating coffee type, followed by “Cappuccino”.
3. A significant portion of total revenue is generated by a small group customers, indicating a dependency on key customer segments.
4. A few coffee types contribute the majority of total revenue, highlighting clear top-performing products that drive overall sales.
5. Revenue shows noticeable variation across months, with peak sales occurring in specific periods, indicating seasonal demand patterns (Highest Sale occurred during the month of MAY).




THANK YOU